CHAPTER 1

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward. Example 1: Input: words = ["abc","car","ada","racecar","cool"] Output: "ada" Explanation: The first string that is palindromic is "ada". Note that "racecar" is also palindromic, but it is not the first. Example 2: Input: words = ["notapalindrome","racecar"] Output: "racecar" Explanation: The first and only string that is palindromic is "racecar".

```
def firstPalindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""
print(firstPalindrome(["abc","car","ada","racecar","cool"]))
print(firstPalindrome(["notapalindrome","racecar"]))
```

```
ada
racecar
```

3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array. Example 1: Input: nums = [1,2,1] Output: 15 Explanation: Six possible subarrays are: [1]: 1 distinct value [2]: 1 distinct value [1]: 1 distinct value [1,2]: 2 distinct values [2,1]: 2 distinct values

[1,2,1]: 2 distinct values The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 + 22 + 22 + 22 = 15. Example 2: Input: nums = [1,1] Output: 3 Explanation: Three possible subarrays are: [1]: 1 distinct value [1]: 1 distinct value [1,1]: 1 distinct value The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 = 3.

```python
def sumCounts(nums):
    total = 0
    n = len(nums)

    for i in range(n):
        distinct = set()
        for j in range(i, n):
            distinct.add(nums[j])
            total += len(distinct) ** 2

    return total
print(sumCounts([1,2,1]))
print(sumCounts([1,1]))
```

```
15
3
```

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2]. Example 1: Input: nums1 = [2,3,2], nums2 = [1,2] Output: [2,1] Explanation: Example 2: Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6] Output: [3,4] Explanation: The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3. The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

```python
def findIntersectionValues(nums1, nums2):
    set1 = set(nums1)
    set2 = set(nums2)

    answer1 = sum(1 for x in nums1 if x in set2)
    answer2 = sum(1 for x in nums2 if x in set1)

    return [answer1, answer2]
print(findIntersectionValues([2,3,2], [1,2]))
print(findIntersectionValues([4,3,2,3,1], [2,2,5,2,3,6]))
```

```
[2, 1]
[3, 4]
```

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same. Test Cases
7. Empty List
8. Input: []
9. Expected Output: None or an appropriate message indicating that the list is empty.
10. Single Element List
11. Input: [5]
12. Expected Output: 5
13. All Elements are the Same
14. Input: [3, 3, 3, 3, 3]
15. Expected Output: 3

```python
def sortAndFindMax(arr):
    if not arr:
        return "List is empty"

    arr.sort()
    return arr[-1]
print(sortAndFindMax([]))
print(sortAndFindMax([5]))
print(sortAndFindMax([3,3,3,3,3]))
```

```
List is empty
5
3
```

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm? Test Cases Some Duplicate Elements Input: [3, 7, 3, 5, 2, 5, 9, 2] Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used) Negative and Positive Numbers Input: [-1, 2, -1, 3, 2, -2] Expected Output: [-1, 2, 3, -2] (Order may vary) List with Large Numbers Input: [1000000, 999999, 1000000] Expected Output: [1000000, 999999]

```python
def uniqueElements(nums):
    seen = set()
    result = []

    for num in nums:
        if num not in seen:
            seen.add(num)
            result.append(num)

    return result
print(uniqueElements([3,7,3,5,2,5,9,2]))
print(uniqueElements([-1,2,-1,3,2,-2]))
print(uniqueElements([1000000,999999,1000000]))
```

```
[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]
```

5. Write a program FOR THE BELOW TEST CASES with least time complexity Test Cases: -

1. Input: {1, 2, 3, 4, 5} Expected Output: 5
2. Input: {7, 7, 7, 7, 7} Expected Output: 7
3. Input: {-10, 2, 3, -4, 5} Expected Output:

```
def findMax(arr):
    maximum = arr[0]

    for num in arr:
        if num > maximum:
            maximum = num

    return maximum
print(findMax([1, 2, 3, 4, 5]))
print(findMax([7, 7, 7, 7, 7]))
print(findMax([-10, 2, 3, -4, 5]))
```

```
5
7
5
```

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

```python
def bubbleSort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr
print(bubbleSort([5,1,4,2,8]))
```

```
[1, 2, 4, 5, 8]
```

10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

```
def mergeSort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2

        left = arr[:mid]
        right = arr[mid:]

        mergeSort(left)
        mergeSort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

    return arr
print(mergeSort([5,2,3,1]))
```

```
[1, 2, 3, 5]
```

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation. Test Case: Example X={ 3,4,6,-9,10,8,9,30} KEY=10 Output: Element 10 is found at position 5 Example X={ 3,4,6,-9,10,8,9,30} KEY=100 Output : Element 100 is not found

```
def binarySearch(arr, key):
    arr.sort()

    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid + 1
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1
arr = [3,4,6,-9,10,8,9,30]

result = binarySearch(arr, 10)
if result != -1:
    print("Element 10 is found at position", result)
else:
    print("Element 10 is not found")

result = binarySearch(arr, 100)
if result != -1:
    print("Element 100 is found at position", result)
else:
    print("Element 100 is not found")
```

```
Element 10 is found at position 7
Element 100 is not found
```